Shuttle Dispatch

Value and Policy Iteration

The infinite-horizon solution is to **wait when fewer than 13 passengers are waiting and dispatch when 13 or more are waiting**. The sections below specify the model, explain each solution method, and present numerical evidence that the implementation is correct.

1 The model: what happens in one period?

Each period has four steps: observe the queue, choose whether to dispatch, charge the dispatch and waiting costs, and then add the new arrivals. The state records only the queue length because the assumed arrival distribution and available actions do not otherwise depend on the past.

Table 1: Notation used to describe the environment.

Symbol	Meaning
t	Decision period; the first decision is at $t = 0$.
s_t	Number of passengers waiting *before* the decision at time t. Possible states are $0, 1, \ldots, 200$.
a_t	Action: 0 means wait; 1 means dispatch one shuttle.
N_t	Random number of new arrivals after the decision. Each of $1, 2, 3, 4, 5$ has probability $1/5$. Arrivals are independent across periods.
$c(s, a)$	Cost of taking action a when the queue contains s passengers.
γ	Discount factor, 0.95. A cost one period later receives weight 0.95 relative to a cost now.

A shuttle can serve 15 passengers. Therefore, after action a in state s, the number still waiting is $\max\{s - 15a, 0\}$. The maximum with zero prevents the queue from becoming negative. The cost is

$$\underbrace{c(s,a)}_{\text{current cost}} = \underbrace{100a}_{\text{dispatch cost}} + \underbrace{2\max\{s-15a,0\}}_{\text{cost of passengers left waiting}} . \tag{1}$$

New arrivals are added after this cost is assessed. The next queue is

$$s_{t+1} = \min\{\max\{s_t - 15a_t, 0\} + N_t,\ 200\}\,. \tag{2}$$

The minimum with 200 enforces the station capacity.

Example. With 20 passengers, waiting costs $2(20) = 40$ and leaves 20 passengers before arrivals. Dispatching serves 15, leaves 5, and costs $100 + 2(5) = 110$. If three passengers then arrive, the next queue is 23 after waiting or 8 after dispatch. Dispatching costs more *now*, but may save enough future waiting cost to be optimal.

Additional assumptions. Only one shuttle can depart per period, with no separate fleet or travel-time restriction. New arrivals first incur waiting cost at the next decision. Overflow above 200 is discarded without an extra penalty. These timing and overflow choices are assumptions, rather than consequences of the queue cap alone. Ties between actions are resolved by waiting.

2 Finite-horizon enumeration: 500 decisions

2.1 The quantity being optimized

Let $J_t(s)$ be the **minimum expected discounted cost still to be paid**, starting with s passengers at time t. Costs are discounted relative to that time. With 500 decisions, its definition is

$$J_t(s) = \min_{\pi} \mathbb{E}_{\pi}\left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s\right]. \tag{3}$$

Here u counts future periods; π denotes a policy, meaning a rule for choosing actions; and $\mathbb{E}_{\pi}$ means an average over arrival randomness when that policy is followed. A smaller $J_t(s)$ is better. The report uses *cost values*; if rewards were defined as negative costs, their values would have the opposite sign.

2.2 Backward induction over states and actions

Part (a)'s "enumeration" is interpreted as evaluating every state, both actions, and all five possible arrivals at each period. There is no terminal penalty, so $J_{500}(s) = 0$. Starting at the last decision and working backward, compute

$$Q_t(s,a) = c(s,a) + \frac{\gamma}{5}\sum_{k=1}^{5} J_{t+1}(\min\{\max\{s-15a, 0\} + k, 200\}), \tag{4}$$

$$J_t(s) = \min_{a\in\{0,1\}} Q_t(s,a). \tag{5}$$

$Q_t(s,a)$ is the expected remaining cost if action a is taken *now* and decisions are optimal afterward. In the sum, k is a possible arrival count. The factor $1/5$ averages the five outcomes, and γ discounts their continuation costs. The minimizing action is saved as $\pi_t(s)$, the decision rule at time t.

This procedure reuses previously computed continuation values, avoiding the exponential cost of listing every complete decision history. The implementation saves all 501 value vectors and all 500 policy vectors.

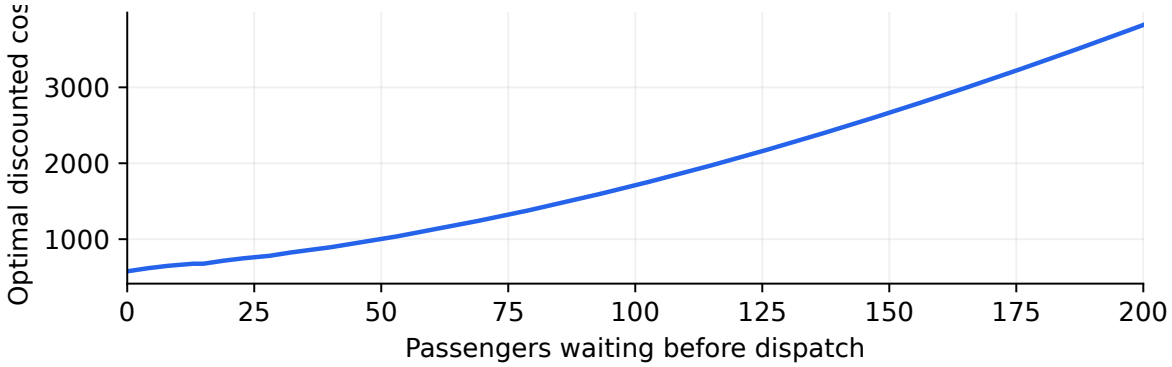


Figure 1: Minimum expected discounted cost at the initial decision, $J_0(s)$, for 500 decisions and zero terminal cost.

The finite-horizon policy need not be stationary: it can change as the end approaches. In the final period, waiting is optimal in every state. A dispatch costs 100 but saves at most $2 \times 15 = 30$ immediately; there is no later benefit or terminal penalty to justify it.

3 Infinite-horizon value iteration

For an unlimited sequence of decisions, write $J^*(s)$ for the minimum expected discounted cost starting from queue s. The superscript $*$ indicates optimality. This value has no time subscript because the problem has no deadline and the environment does not change over time.

3.1 The Bellman update

Starting from $J^{(0)}(s) = 0$, repeatedly compute

$$J^{(n+1)}(s) = \min_{a\in\{0,1\}} \left[c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J^{(n)}(\min\{\max\{s-15a,0\}+k, 200\}) \right]. \tag{6}$$

The superscript (n) counts algorithm iterations, not decision periods. Each update asks: *which action has the lowest current cost plus discounted expected future cost?* Its structure is the same as the finite-horizon recursion, but it improves an estimate of the stationary value instead of moving backward through time.

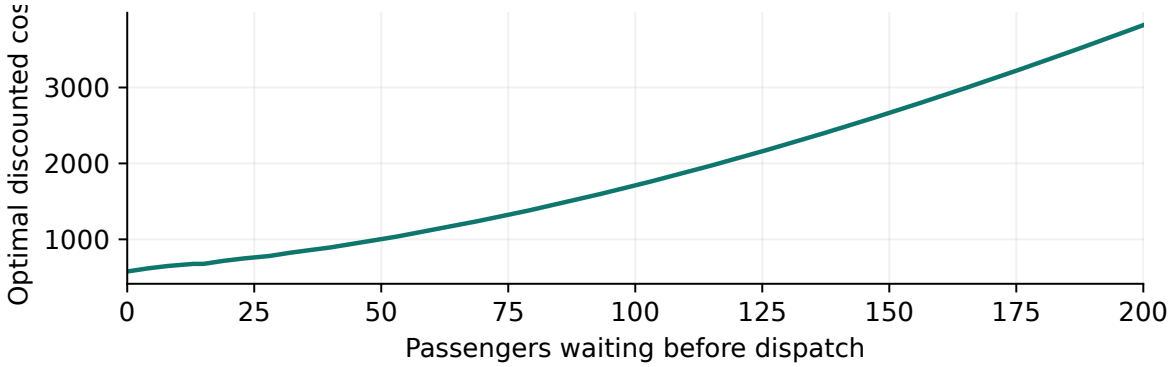


Figure 2: Optimal infinite-horizon cost $J^*(s)$, estimated by value iteration. Larger initial queues incur greater total expected cost.

3.2 When does iteration stop?

Let $\mathcal{T}J$ denote the entire right-hand side of Equation (6), evaluated using a candidate value vector J. This is the *Bellman operator*: one application performs one value update. Define its residual as

$$\delta(J) = \max_{s\in\{0,\ldots,200\}} |(\mathcal{T}J)(s) - J(s)|. \tag{7}$$

The residual measures how much another Bellman update would change the candidate values, in the worst state. The algorithm stops when $\delta(J) \le 10^{-10}$, which required **518 updates**.

Because $\gamma < 1$, the Bellman operator contracts maximum errors by at most γ. Consequently,

$$\max_s |J(s) - J^*(s)| \le \frac{\delta(J)}{1-\gamma}. \tag{8}$$

This connects a computable stopping criterion to the accuracy of the value estimate.

The implementation minimizes costs. Thus, its arrays named `finite_costs`, `vi_cost`, and `pi_cost` are the cost values J_t and J^* described above.

4 Policy iteration and the dispatch rule

Policy iteration alternates between evaluating a fixed decision rule and improving it. It starts from the rule "always wait."

Policy evaluation. For a stationary policy π, $\pi(s)$ is its action in state s. Let J_π be the vector of expected discounted costs when that rule is followed indefinitely. Let P_π be the 201×201 transition matrix: entry (s, s') is the probability of next queue s' after taking action $\pi(s)$. Let c_π be the vector with entries $c(s, \pi(s))$. Then

$$J_\pi = c_\pi + \gamma P_\pi J_\pi, \qquad \text{so} \qquad (I - \gamma P_\pi) J_\pi = c_\pi. \tag{9}$$

Here I is the identity matrix, and s' denotes a possible next state. The linear system computes the cost of the current policy. When several arrival counts produce the same next state, their probabilities are added in P_π.

Policy improvement. For every state, compare both actions using J_π as the continuation value in Equation (6). Replace the current action with the lower-cost one. Stop when no actions change. This took **six evaluation/improvement rounds**, including the final stability check.

The resulting optimal stationary rule is

$$\boxed{\pi^*(s) = \begin{cases} 0 & \text{(wait)}, \quad 0 \le s \le 12, \\ 1 & \text{(dispatch)}, \quad 13 \le s \le 200. \end{cases}} \tag{10}$$

The threshold was discovered by comparing both actions in every state; it was not imposed on the search. Departing with 13 or 14 passengers can be worthwhile because the savings in future holding costs outweigh the benefit of waiting for a full shuttle.

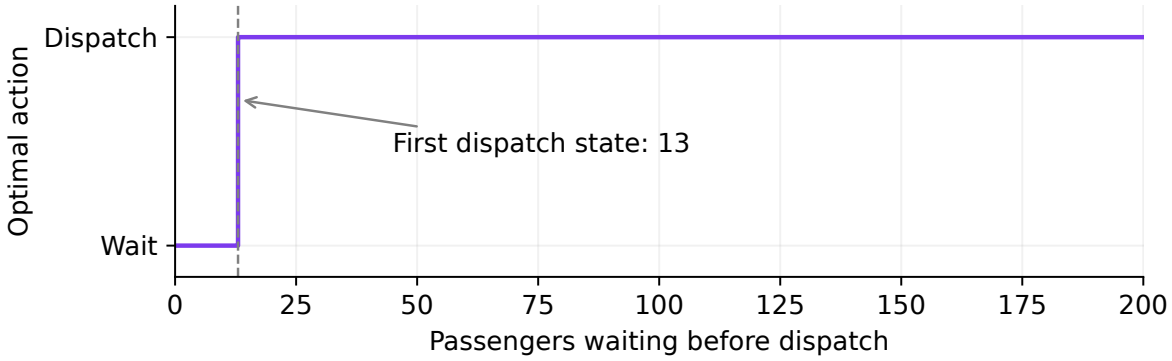


Figure 3: Optimal stationary policy from policy iteration. The first dispatch state is 13.

Table 2: Representative minimum infinite-horizon discounted costs.

Initial passengers s	Optimal cost $J^*(s)$
0	576.670168
15	676.670168
200	3821.586477

The difference $J^*(15) - J^*(0) = 100$ is a useful consistency check. Dispatch at 15 clears the queue, while waiting at zero also leaves it empty. Both lead to the same next-arrival distribution; only the dispatch charge differs.

5 Edge cases and correctness checks

All **14 validation checks passed**. The checks use optimality conditions, independent solution procedures, and model changes with predictable outcomes.

5.1 Agreement and optimality

Value iteration and policy iteration selected identical actions in all 201 states. Their maximum value difference was 1.964×10^{-9}. The policy-iteration result had Bellman residual 9.095×10^{-13}. Equation (8) therefore bounds its maximum value error by approximately 1.819×10^{-11}, subject to floating-point rounding. The smallest absolute difference between the two action costs was 3.778, well above the numerical error.

The maximum difference between the 500-period time-zero values and infinite-horizon values was 4.942×10^{-9}. Since the largest possible stage cost is 470, a conservative bound on the discounted cost beyond 500 periods is

$$\sum_{t=500}^{\infty} \gamma^t \, 470 = \frac{470\gamma^{500}}{1-\gamma} = 6.838 \times 10^{-8}. \tag{11}$$

The observed difference is smaller than this bound. Agreement is expected because very late costs receive little weight under discounting.

5.2 Queue boundaries and the final period

Service never produces a negative queue: 15 passengers clear exactly, and 16 leave one. At state 200, waiting always gives next state 200; dispatch gives next states 186 through 190. At zero, dispatch costs 100 more than waiting without changing the next-state distribution, so it is strictly dominated. These cases verify that service, saturation, and probability pooling are implemented consistently.

The terminal value vector was checked to be zero. The last-period policy was also checked against the analytic prediction that waiting is optimal everywhere. This tests that the finite-horizon algorithm does not charge continuation costs beyond its deadline.

Table 3: Model modifications with known answers. In this table, $J(s)$ denotes the optimal cost for the modified problem.

Modification	Predicted result	Observed result
Zero holding cost	Always wait; $J(s) = 0$.	Matched all states.
Zero shuttle capacity and no arrivals	Always wait; $J(s) = 2s/(1-\gamma)$.	Zero analytic error.
Free dispatch, capacity 200, no arrivals	Dispatch at nonempty states; $J(s) = 0$.	Matched all states.
Zero discount factor	Only current costs matter: always wait, with $J(s) = 2s$.	Matched all states.

A separately coded exhaustive decision/arrival tree for a seven-state, four-period example matched backward induction exactly. Complete enumeration is feasible at this scale and provides an independent check of the finite-horizon recursion.

6 Independent simulation and reproducibility

The optimal policy was simulated from initial queues of 0, 15 and 200. Each starting state used 20,000 independent trajectories of 300 decisions, with random seed 1668. A trajectory's total cost is the sum of its stage costs weighted by $1, \gamma, \gamma^2, \ldots$. The simulator calculated transitions and costs directly instead of reusing the solver's transition-array builder.

Table 4: Simulation estimates compared with exact policy evaluation.

Initial queue	Simulation mean	Exact cost	Standard error
0	576.671469	576.670168	0.219213
15	676.814669	676.670168	0.221520
200	3821.605190	3821.586477	0.532280

The standard error estimates uncertainty in the simulation *mean*: it is the sample standard deviation of trajectory costs divided by $\sqrt{20{,}000}$. Each mean was within four standard errors of the exact value, allowing an additional 0.001951 for the conservative bound on costs omitted after period 300. This wider tolerance is the automated simulation check; Figure 4 displays approximate 95% intervals using ± 1.96 standard errors.

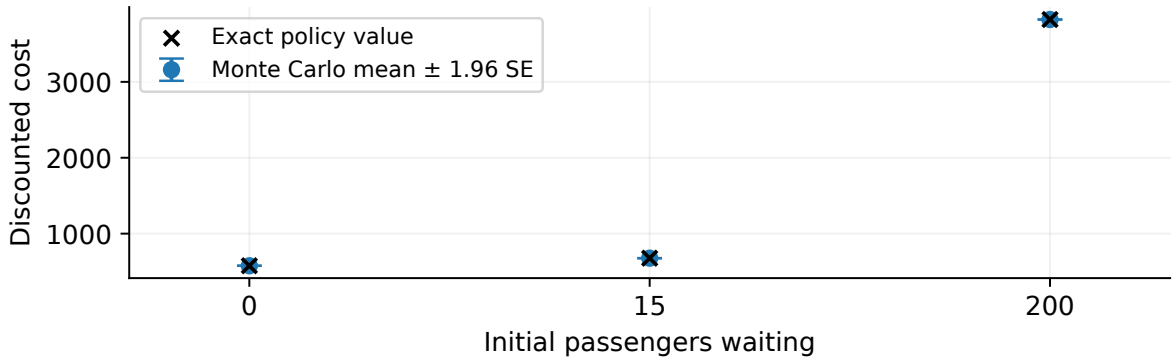


Figure 4: Independent simulation means and exact policy costs. Error bars describe uncertainty in the mean, not the variation of individual trajectory costs.

Simulation supports the computed policy values but does not prove optimality. The Bellman residual provides the stronger optimality check. Likewise, algorithm agreement alone cannot catch every modeling mistake because the methods share the same specification; the analytic examples and independently coded tree and simulator provide additional evidence.

6.1 Reproducing the results

From the project folder, run:

```
python -m pip install -r requirements.txt
python shuttle.py
jupyter lab shuttle_plots.ipynb
```

The solver runs all three methods and all validation checks, raising an error if any check fails. Full evidence is saved in `results/summary.json`; per-state results are in `results/state_results.csv`. The saved arrays in `results/shuttle_results.npz` include every finite-horizon value and policy. Running all notebook cells regenerates the assignment plots in `results/figures/`. For the compact vector figures used in this report, run `make_report_figures.py`; then compile `SHUTTLE_REPORT.tex`.

7 Validation with an additional three months

An extended study would focus on three questions.

How sensitive is the dispatch rule to the parameters?

Vary dispatch and holding costs, shuttle capacity, discounting, and arrival distributions. Record changes in the optimal threshold and test whether threshold behavior persists. Compare larger queue caps, such as 250, 500 and 1,000, and introduce penalties for lost arrivals. These experiments would reveal whether the 13-passenger recommendation depends strongly on the chosen parameters or overflow assumption.

Can another method confirm the solution?

Construct an independent linear-programming formulation and compare its values and actions with dynamic programming. Use higher precision when action costs are nearly tied. Vary the finite horizon and terminal penalties to study decisions near the deadline. Use more simulation seeds and starting queues, and measure how often nominal confidence intervals contain the exact policy value.

Does the policy remain effective under more realistic conditions?

Replace independent uniform demand with bursty or correlated arrivals, and evaluate the original policy under these alternative demand processes. Add shuttle availability and travel times to the state. These extensions would test robustness to modeling assumptions, beyond verifying numerical correctness for the stated environment.

Assignment 1: Policy Gradient

CartPole-v1 and Pong-v5, Parts 1 and 2

GitHub username: FailedMath3U **Repository:** MLDS
`https://github.com/FailedMath3U/MLDS`

1 Algorithm and experimental protocol

The PyTorch implementation uses REINFORCE: collect complete episodes, compute discounted reward-to-go, then update the policy once per batch. The policy $\pi_\theta(a_t \mid o_t)$ gives action probabilities from network input o_t; θ denotes all actor weights and biases. At decision t, the sampled action is a_t and the subsequent reward is r_t. For episode length T and discount γ, optimize

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

G_t is the discounted reward remaining from decision t, calculated backward with zero continuation at the episode end. For B episodes, Adam minimizes

$$\mathcal{L} = -\frac{c}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(a_t^{(i)} \mid o_t^{(i)})(G_t^{(i)} - b_t^{(i)}).$$

Here i indexes episodes, b_t is the baseline (zero in Part 1), and c is a fixed positive scale. The code sums decision contributions and divides by the number of episodes, not their total length. Returns and baseline predictions are held fixed during actor differentiation. The two output logits define a categorical action distribution. There is no return normalization or centering. Pong accumulates gradients in chunks but takes only one optimizer step per complete batch.

Actor setting	CartPole	Pong
Network	$4 \to 64$ tanh $\to 2$ logits	$6400 \to 200$ ReLU $\to 2$ logits
Discount γ; loss scale c	0.95; 0.05	0.99; 1
Adam learning rate	0.01	0.001
Episodes per update; total per seed	32; 8,000	8; 20,000
Training seeds	1668, 1669, 1670	1668
Gradient-norm cap	10	5
Hardware	CPU	One GPU

Evaluation protocol. Each required histogram uses 500 fresh episodes from the final fixed-budget policy, with no learning. Actions are sampled, not chosen greedily. Test reset seeds are separate from training and shared across methods. CartPole seed 1668 was fixed before the full runs for the main plots; all three seeds are reported. Every figure uses undiscounted episode reward $\sum_t r_t$; SD is the population standard deviation across its 500 evaluations.

Alternative Pong learning rate. On identical real-game data, one update at 0.001 produced mean action-distribution KL divergence 1.996×10^{-6}, versus 3.423×10^{-4} at 0.01. This motivated a smaller step; it does not prove greater long-run stability. Clipping affected 61.68% of basic Pong updates and 18.64% with a baseline. No recorded CartPole update reached its cap.

2 Part 1(a): CartPole-v1 without a baseline

Model. The four inputs are cart position, cart velocity, pole angle and pole angular velocity. Actions 0 and 1 apply left and right force. The default reward is +1 per transition, including failure. Physical failure occurs beyond cart position ± 2.4 or pole angle $\pm 12°$; a time limit stops episodes at 500 steps. Each reset independently samples the four components within $(-0.05, 0.05)$ using recorded episode seeds. We optimize this capped episode, so returns stop at either end flag without bootstrapping. The critic in Part 2 additionally observes time; the actor does not.

Result. The primary final policy scored **500.000 ± 0.000**. All 500 evaluations reached the limit without physical failure. An untrained network scored 20.926±11.467 and a uniform random policy 22.782±12.058 over 500 episodes each. Learning was substantial but not stable throughout training; Section 8 tests the large dips directly.

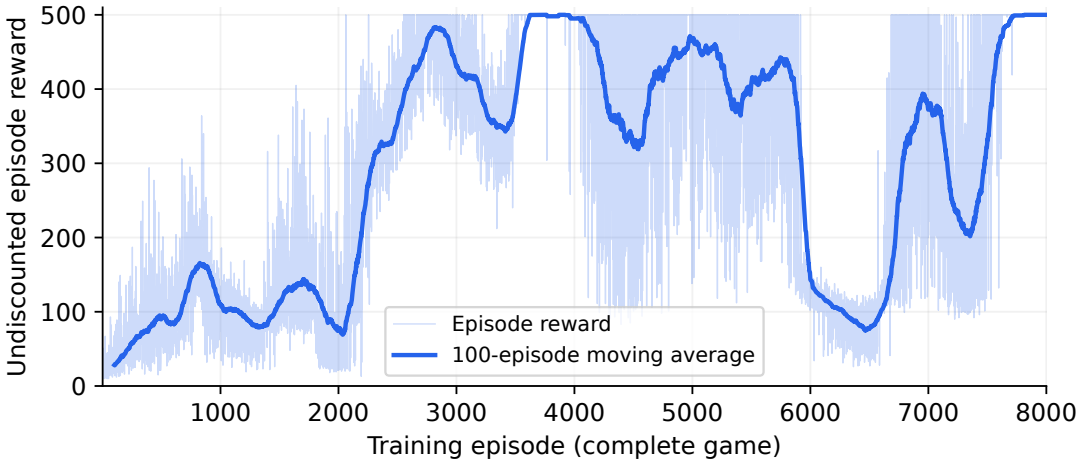


Figure 1: CartPole, Part 1: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 complete episodes.

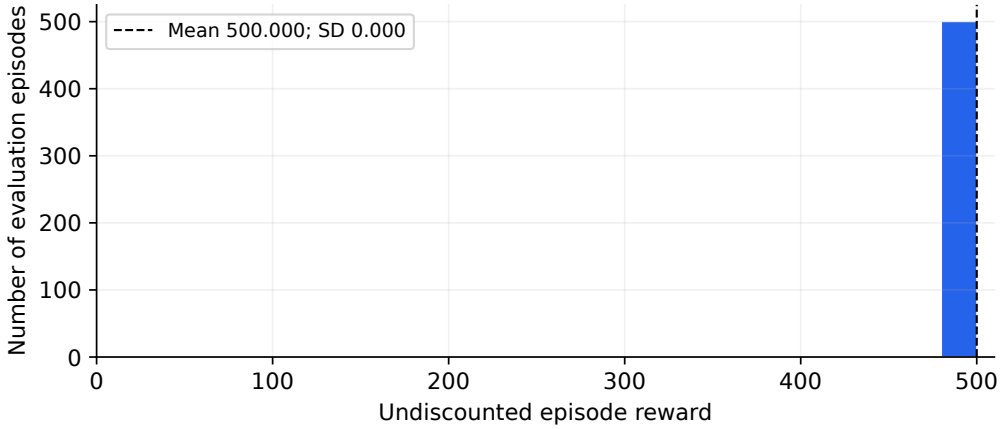


Figure 2: CartPole, Part 1: 500 final-policy evaluation rewards. Mean 500.000; population SD 0.000.

3 Part 1(b): Pong-v5 without a baseline

Model. We use `ALE/Pong-v5`, the namespaced Pong-v5 environment, with only actions 2/3 (ALE labels RIGHT/LEFT; vertical paddle movement). Rewards are +1 when the agent scores, −1 when the opponent scores, and zero otherwise. Episodes are complete games, not points. The assignment's crop, factor-two downsampling and background removal produce binary 80×80 frames. The network receives the flattened signed difference of consecutive frames; the first input after reset is zero.

This is a reactive policy in a partially observable process: o_t supplies motion cues but is not the full emulator state s_t. Unchanged image regions cancel, and the network has no recurrent memory. Transition probabilities are supplied implicitly by the emulator, not learned. Defaults retain four emulator frames per decision, sticky-action probability 0.25 and a 108,000-frame cap.

Result. The final policy scored **-18.610 ± 1.681**, versus an untrained reference of -20.250 ± 0.935 over 32 different games. All 500 test games were losses (range −21 to −10), with no truncations. **The policy did not learn to win.** The reference comparison is descriptive, not a paired test.

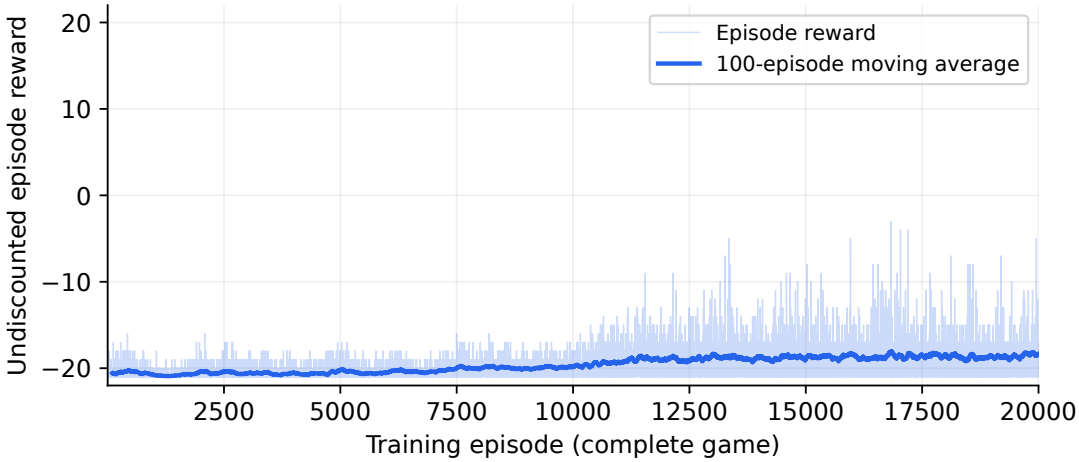


Figure 3: Pong, Part 1: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.



Figure 4: Pong, Part 1: 500 final-policy evaluation rewards. Mean -18.610; population SD 1.681.

4 Part 2(a): CartPole with a learned baseline

Choice and implementation. A separate network with inputs $(o_t, t/500)$, 64 tanh units and one output predicts G_t. Time matters because fewer rewards remain near the episode limit. Before each actor update, its predictions are computed without gradients. Afterward, five Adam steps at rate 0.01 fit the observed Monte Carlo returns by mean squared error. There is no temporal-difference bootstrap. Actor settings, initial actor weights and episode budgets match Part 1.

Subtracting an action-independent baseline preserves the expected *unclipped* policy gradient:

$$\mathbb{E}_{a\sim\pi_\theta}[b\nabla_\theta \log \pi_\theta(a \mid o)] = b\nabla_\theta \sum_a \pi_\theta(a \mid o) = 0.$$

Here ∇_θ is the vector of derivatives with respect to actor parameters. This identity does not guarantee lower variance for an inaccurate baseline.

Result. The final model scored **260.308 ± 57.013**: four episodes reached 500, and 496 ended in physical failure. It improved over random control but underperformed the primary basic policy. The variance experiment in Section 6 distinguishes a baseline's local variance benefit from successful training.

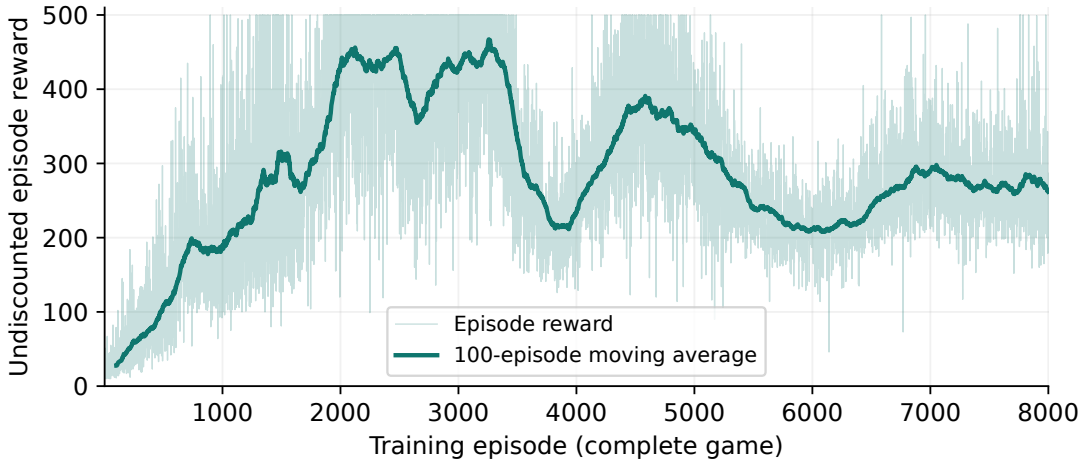


Figure 5: CartPole, Part 2: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 complete episodes.

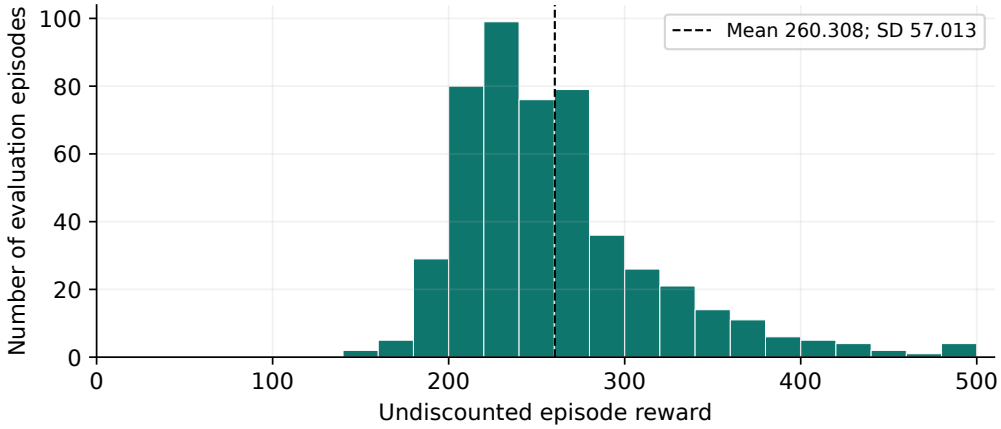


Figure 6: CartPole, Part 2: 500 final-policy evaluation rewards. Mean 260.308; population SD 57.013.

5 Part 2(b): Pong with a constant baseline

Choice and implementation. The baseline is a scalar exponential moving average (EMA), held constant within each batch. Starting at $b_0 = 0$, update k uses only the previous value; afterward,

$$\bar{G}_k = \frac{\sum_{i,t} \gamma^t G_t^{(i)}}{\sum_{i,t} \gamma^t}, \qquad b_{k+1} = 0.95 b_k + 0.05 \bar{G}_k.$$

k counts actor updates. Thus the current sampled actions do not determine the baseline used for their own gradient. Apart from this subtraction, the actor architecture, initial weights, environment, budgets and action restrictions match Part 1.

Result. The final model scored **-18.446 ± 1.851**, 0.164 above the basic method. All 500 games were losses (range −21 to −5), with no truncations. One training seed does not establish a general benefit. Later paired probes found no consistent variance reduction from the actual EMA (Section 7). Frequent gradient clipping also means actual updates differ from the unclipped estimator used in the baseline identity.

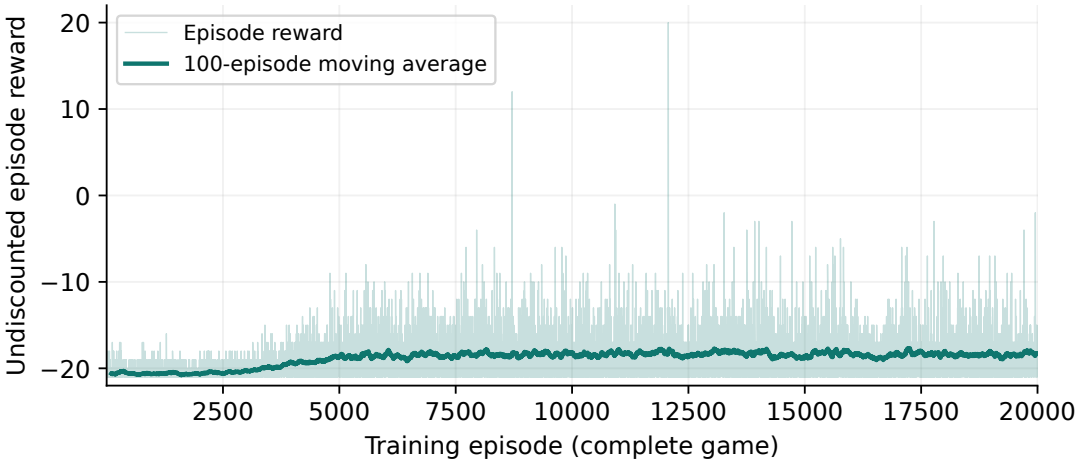


Figure 7: Pong, Part 2: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.



Figure 8: Pong, Part 2: 500 final-policy evaluation rewards. Mean -18.446; population SD 1.851.

6 CartPole: edge cases and validation

Boundary crossings and unavoidable failure. Supplementary checks tested both actions at both signed cart/pole limits, and just inside each limit with outward velocity: all 16 predictions matched. Equality at the threshold is allowed; crossing it terminates. For example, $x = 2.39$ and $\dot{x} = 1$ give $x' = 2.41$ under either action because the default Euler step uses $x' = x + 0.02\dot{x}$. Force changes velocity too late to avoid that transition's failure. The terminal reward remained +1.

Failure and time limit together. Setting a one-step limit and forcing a boundary crossing returned both `terminated=True` and `truncated=True`, with one reward of 1. A score of 500 alone therefore cannot identify failure-free survival; both flags are retained. The separate original time-limit check verified truncation without failure.

Different episode lengths. The actual collector was exercised with controlled episodes of lengths 1, 1 and 3. It recorded no post-end transitions; the one-step return was exactly 1. The combined loss gradient agreed with the mean of separately computed episode gradients to a maximum component error of 1.863×10^{-9}. This checks trajectory separation and averaging by episode rather than introducing length-dependent normalization.

Negative controls and recovery limits. Frozen primary actors and fixed-action controls were evaluated on 64 matched starts per condition, without retraining. Trained actors used *greedy* actions here, separately from the required stochastic evaluations:

Policy	Default resets	Tilted, moving outward	Near track edge
Always left	9.34	9.50	7.75
Always right	9.38	9.50	7.78
Basic, greedy	469.77	488.44	8.70
Baseline, greedy	283.61	284.77	13.33

Entries are mean episode rewards. Tilted starts set pole angle to ± 0.15 radians and angular velocity to ± 0.5; edge starts set cart position to ± 2.3 and velocity to ± 0.5. Signs alternate; other components retain seeded reset values. Fixed-direction controllers fail quickly, while trained policies balance from default and tilted starts. Both struggle near the track edge. These exploratory stress tests expose limits of standard-reset results; they do not establish that every edge start is recoverable. The basic actor's greedy default score also shows why evaluation action selection must be specified.

Baseline variance. At a frozen initial actor, a separate critic fitted on 512 trajectories was measured on 512 different, paired trajectories. Summed sample variances of gradient components fell from 5.338262 to 0.842421 (84.22%). The mean-gradient difference was 0.0474 versus paired standard-error magnitude 0.1176. This supports variance reduction at that actor, but the critic received 150 fitting steps; it does not establish the same benefit for five-step online fitting throughout training.

Decision inspection. The saved ten-step trace contains all four observations, probabilities, sampled actions and rewards. At step 3, right had probability 0.7711 but left was sampled; the pole subsequently remained within bounds. This is compatible with stochastic action selection, not evidence of a reversed action mapping. A separate fixed-seed, ten-action replay reproduced identical state sequences.

7 Pong and shared gradient validation

Scoring is not an episode end. For rewards $(0, 1, 0, -1)$ at $\gamma = 0.99$, independently calculated returns are

$$(G_0, G_1, G_2, G_3) = (0.019701, 0.0199, -0.99, -1).$$

The implementation matched within 10^{-6}. In particular, $G_1 = 1 - 0.99^2$ includes the later loss; incorrectly restarting returns at the positive point would give 1. Complete-game collection was checked separately. Both end flags stop collection, and frame history is reset between games.

Serving under restricted actions. Eight complete preflight games using only actions 2/3 each produced 21–22 scoring events and reached natural termination. No startup FIRE/NOOP wrapper was added. This verifies progress under the assigned restriction, not that FIRE has no effect in unrestricted play.

Sticky actions. The emulator can repeat its previously executed action with probability 0.25 per emulator frame. Requested and executed actions therefore need not coincide throughout a four-frame decision. We retain this stochasticity and test seeded replay, rather than assuming deterministic action execution: identical reset seeds and requested-action sequences reproduced frame/reward hashes. This establishes reproducibility, not robustness across all seeds.

Visual input. Known-pixel images verified the crop, factor-two sampling, background removal and foreground preservation. Subtraction uses signed values, so disappearing pixels become -1 rather than unsigned overflow. Real differences were in $\{-1, 0, 1\}$; reset input was zero. The saved game trace includes the first ten decisions, a point and the ending, with corresponding image differences. Its positive point at decision 93 and final loss at 1,797 illustrate how rewards can be far apart within one episode.

Does the Pong baseline reduce variance? An independently fitted candidate constant reduced summed gradient variance from 369.320 to 20.763 on 16 held-out games (94.38%). It was fitted on eight other games and was *not* the initial training EMA, which starts at zero. Later probes used the actual frozen EMA and 16 paired games at the same actor. Variance ratios (with/without baseline) at 4,096, 8,192, 12,288 and 16,384 games were 0.873, 1.017, 0.924 and 1.011. Reduction was not sustained. Small probe samples and the absence of ratio confidence intervals limit precision; all probes measure unclipped gradients.

Analytic gradient controls for both parts. In a two-step task with Bernoulli action probability $p = \text{sigmoid}(\theta)$ and reward equal to the action, the exact objective is $(1 + \gamma)p$. Enumeration over all four action sequences reproduced its derivative $(1 + \gamma)p(1 - p)$, with and without a constant baseline: error 8.52×10^{-9} for CartPole's discount and below 10^{-7} for Pong's. Setting discount to zero recovered immediate rewards. Zero Pong returns produced zero basic-policy gradient. A one-step actor check with target 1 changed the selected action's probability from 0.5 to 0.505000 with baseline 0, and to 0.495000 with baseline 2. This verifies both signs of the advantage, rather than assuming positive game rewards require every action probability to increase.

These controls test the estimator and task handling. They do not establish that the Pong representation, optimizer settings or training budget were adequate to learn winning play.

8 Tuning and interpretation

All CartPole seeds. Each row is a final policy evaluated on 500 stochastic episodes at the original actor rate 0.01:

Method	Seed	Mean	Population SD	Fraction at 500
No baseline	1668	500.000	0.000	1.000
No baseline	1669	271.824	28.856	0.000
No baseline	1670	483.886	65.298	0.928
Learned baseline	1668	260.308	57.013	0.008
Learned baseline	1669	360.516	79.141	0.100
Learned baseline	1670	349.940	76.321	0.092

The baseline improves seed 1669, but not the other two. The following additional runs changed only actor rate, keeping seeds, 8,000-episode budgets and 500 evaluations per model; critic rate remained 0.01:

Actor rate	Basic: mean across three seeds	Baseline: mean across three seeds
0.01	418.570	323.588
0.005	163.178	357.661
0.002	247.102	264.481

Smaller rates did not consistently improve final reward. These exploratory comparisons do not identify an optimal rate or establish that a baseline generally helps.

Are the training dips just episode noise? Both seed-1668 runs were replayed exactly: every original training reward and final parameter hash matched. Before/during checkpoints selected from those curves were then frozen and evaluated on 500 fresh paired reset/action seeds:

Method	Before mean	During mean	Change [paired 95% interval]
Basic (5,120 → 6,400)	453.108	83.114	−369.994 [−377.430, −362.038]
Baseline (3,072 → 6,144)	440.466	215.686	−224.780 [−231.886, −217.436]

Intervals use 5,000 paired bootstrap replicates. A further 2,000 episodes with each earlier actor frozen gave 100-episode mean ranges of 430.11–472.29 (basic) and 413.00–453.82 (baseline); continued training reached 74.64 and 211.37. Thus the selected dips reflect policy changes beyond measured fixed-policy episode noise. Noisy updates may still cause those changes. Selection was exploratory; intervals condition on these actors and do not measure uncertainty across training seeds. Rolling windows overlap, so their ranges are descriptive.

Objective versus displayed score. For CartPole, discounted return is $(1 - 0.95^T)/0.05$, close to 20 long before 500 steps. The primary baseline's mean discounted return was 19.999743 despite its lower survival time. In Pong, mean discounted return improved from −1.309180 for the untrained reference to 0.313913 (basic) and 0.293346 (baseline), while all final games were losses. A reward 1,000 decisions away has weight $0.99^{1000} \approx 4.32 \times 10^{-5}$. This suggests a credit-assignment difficulty; it does not identify the cause of failure. Partial observation, optimization and the single training seed remain unresolved limitations.

9 Execution and evidence

Paths are relative to the repository's `assignment1/`. Retrain in a fresh copy to preserve saved runs. Versions: Python 3.12, PyTorch 2.14.0, NumPy 2.5.3, Gymnasium 1.2.2; Pong also uses ALE 0.11.2.

CartPole:

```
cd mlds_assignment1_cartpole
python -m pip install -r requirements.txt
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
python validate_task_edges.py
python instability_experiment.py
python plot_instability_experiment.py
```

The suite trains/evaluates six actors. `validate.py` writes analytic-gradient, return and baseline-variance evidence to `validation/`. `validate_task_edges.py` writes boundary, collector, advantage-sign and recovery evidence to `task_validation/`. The instability scripts reproduce replay, frozen-policy comparisons and bootstrap results. Each output folder contains numerical records; the original notebook plots saved runs. Lower-rate folders `cartpole_lr005/` and `cartpole_lr002/` provide `run.sh` for their complete suites.

Pong: install its requirements and copy source/config/supervisor into a fresh directory. Set one GPU UUID in `config.json` and `supervise.sh`, and configure the supervisor's Python executable. Activate that environment:

```
export CUDA_VISIBLE_DEVICES=<single-GPU-UUID>
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python run_pong.py preflight
nohup setsid bash supervise.sh > nohup.log 2>&1 < /dev/null &
```

Preflight runs the 15 checks; training alternates methods on one GPU and probes variance every 4,096 games. Existing checkpoints resume. Evidence: `preflight/results.json`, `runs/*/training.csv`, `runs/*/evaluation/test_final.json` and `diagnostics/`. The README explains recovery and traces. The original suite took 2.64 hours; CartPole used CPU only.

Report: run `python build_assets.py`, then compile this LaTeX source. The builder audits numerical results and regenerates figures. `python combine_submission.py` adds the shuttle PDF.

10 Experiments with three additional months

CartPole: use more training seeds and map recovery across initial positions, angles and velocities. Compare actor rate, batch size and critic-fitting frequency under matched budgets. Measure critic error and paired gradient variance throughout training to test whether critic lag explains instability.

Pong: keep v5, actions 2/3 and $\gamma = 0.99$ while comparing frame differences, frame stacks and a convolutional policy. Measure gradient contributions by decision time. Tune rate, clipping and batch size on validation seeds; evaluate multiple training seeds on untouched 500-game tests.

Baselines: compare scalar, state-dependent and time-dependent predictions at identical frozen actors with independent fitting data. Use hundreds of paired games, variance-ratio confidence intervals, and separate clipped/unclipped measurements to test whether variance reduction improves control.